Multilingual Model for Text Processing Languages of India

Harika

January 11, 2026

Abstract

This report presents a unified multilingual text-processing system that performs transliteration, text normalization, and punctuation restoration across a wide range of Indian languages. Our transliteration component covers all 22 scheduled languages of India, while normalization is trained on 12 languages, and punctuation restoration on 23 languages. We construct a large training corpus by combining public datasets with synthetic data generated using Gemini-2.0-Flash. The normalization pipeline verbalizes numbers, dates, and abbreviations in English and is then translated into target languages to form parallel pairs. Additional corpora were transliterated using Gemini-2.0-Flash to ensure full script coverage, and punctuation restoration data was created using aligned punctuated–unpunctuated text. Fine-tuning the Gemma-3-1B-PT model jointly on all tasks enables a single system capable of script conversion, structural correction, and punctuation recovery across diverse languages. The unified model achieves strong multilingual performance and surpasses existing baselines such as the Sarvam Transliteration API and Indic-xlit Engine.

1 Introduction

In today's rapidly evolving digital ecosystem, multilingual content is increasingly prevalent especially within the Indian context, where 22 scheduled languages are officially recognized: Assamese, Bengali, Bodo, Dogri, Gujarati, Hindi, Kannada, Kashmiri, Konkani, Maithili, Malayalam, Manipuri (Meitei), Marathi, Nepali, Odia, Punjabi, Sanskrit, Santali, Sindhi, Tamil, Telugu, and Urdu. The coexistence of many writing systems introduces practical challenges for search, information retrieval, annotation, accessibility, and the development of inclusive NLP technologies. Transliteration, text normalization, and punctuation restoration together form a critical pre-processing stack: transliteration aligns scripts into a common representation; normalization converts written tokens (e.g., numbers, dates, addresses, abbreviations) into consistent, human-readable forms; and punctuation restoration recovers structural cues that are essential for readability and downstream modeling.

This report presents the design, training, and evaluation of a unified multilingual text-processing system built on **Gemma-3-1B-PT**. At its core, the system rests on two pillars: a *high-quality, carefully curated dataset* spanning diverse domains and scripts, and a *rigorous training regimen* featuring multi-task fine-tuning, strong regularisation, and robust

validation checkpoints. The system jointly addresses three tasks at scale: (i) *transliteration* for all **22** scheduled Indian languages, (ii) *text normalization* for **12** languages, and (iii) *punctuation restoration* for **23** languages. We curate a large, high-quality training corpus by combining existing transliterated resources with additional corpora that we transliterate and augment using Gemini-2.0-Flash, followed by strict quality control (deduplication, script normalisation, and heuristic filtering). For normalization, we construct a rule-guided pipeline that verbalizes numerals, expands abbreviations, and standardizes formats for dates, addresses, and other structured entities; we then translate normalized English data into target languages to build parallel supervision. For punctuation restoration, we pair unpunctuated text with gold references to teach the model to recover syntactic and prosodic boundaries.

Our unified approach offers three practical advantages. **First**, a shared architecture reduces engineering overhead compared to maintaining three separate models, while enabling multi-task transfer across closely related signals (script mapping, lexical standardization, and sentence structuring). **Second**, a common Latin (English) transliteration layer supports interoperable tooling for search, labeling, deduplication, and quality control across scripts. **Third**, large-scale augmentation via Gemini-2.0-Flash improves coverage and robustness for low-resource settings without sacrificing consistency.

We evaluate the system using character- and word-level error rates for transliteration/normalization quality and BLEU for sequence fidelity, alongside targeted human spot-checks for edge cases (code-mixing, named entities, numeral phrases). Results indicate that our transliteration component achieves **state-of-the-art** performance across the 22 scheduled languages and consistently **outperforms** competitive baselines such as the Sarvam Transliteration API and Indic-xlit Engine, while the normalization and punctuation modules deliver strong accuracy across their respective language sets. Taken together, these findings position the proposed model as a scalable, reliable foundation for Indian-language NLP pre-processing and cross-script applications.

2 Related Works

transliteration: sarvam-transliterate, indic-xlit-engine

3 Datasets

To develop and evaluate our multilingual transliteration models, we constructed a comprehensive dataset spanning all 22 scheduled Indian languages. Our data pipeline combined both existing transliterated corpora and additional resources which we transliterated ourselves using the Gemini-2.0-Flash model. This approach ensured broad language coverage, high script consistency, and robust training data even for low-resource scenarios.

3.1 Transliteration

Existing Corpora with Native Transliteration

The following datasets already contained high-quality transliterated data and were directly incorporated into our training and evaluation pipeline:

Updesh-beta A multilingual question-answering and reasoning dataset, containing native transliterations across languages [5].

IndicAlign Part of the IndicLLMSuite, providing pre-training and fine-tuning data with native transliterations [3].

Corpora Transliterated Using Gemini-2.0-Flash

To expand the diversity and scale of our dataset, we further leveraged the Gemini-2.0-Flash model to transliterate additional corpora which were originally in native scripts. These resources include:

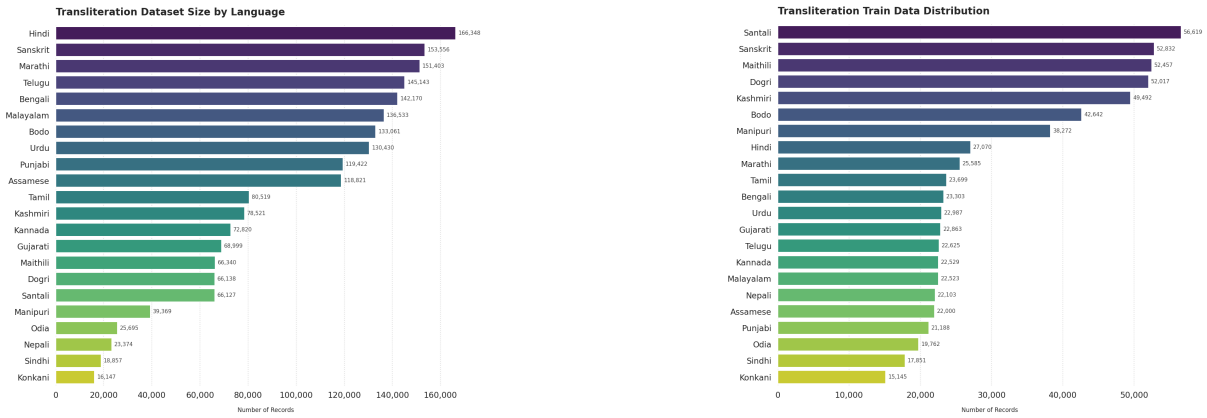
Transliterated Sangraha Based on the IndicLLMSuite, transliterated by us using Gemini-2.0-Flash [3].

Transliterated IndicCorp-v2 A large-scale monolingual corpus, transliterated by us using Gemini-2.0-Flash [1].

Transliterated Cosmopedia-Translated A multilingual punctuation and speech corpus, transliterated by us using Gemini-2.0-Flash [4].

Transliterated BPCC The Bharat Parallel Corpus Collection, transliterated by us using Gemini-2.0-Flash [2].

By integrating both existing transliterated resources and newly transliterated corpora, we ensured a rich and comprehensive dataset foundation for effective supervised fine-tuning and benchmarking of our transliteration models.



(a) The data that we have collected

(b) The data we actually used

Figure 1: Transliteration data

3.2 Data Generation and Normalization

A structured multi-stage pipeline was adopted to create a realistic and high-quality normalization dataset. The process began with the generation of unnormalized English text using the **Gemini 2.0 Flash** for normal languages and for scarce or low resource languages we used **Gemini 2.5 Flash** large language model. A carefully designed system prompt was used to ensure that the generated text closely resembles real-world noisy and informal language rather than grammatically perfect sentences.

The unnormalized data spans multiple categories, including conversational text, informal messaging, abbreviations, numerals, inconsistent casing, spacing irregularities, and punctuation noise. This diversity was intentionally introduced to reflect practical normalization challenges.

3.2.1 System Prompt for Data Generation

The system prompt provided detailed and strict instructions governing linguistic style, variability, and formatting, making it a critical component of the dataset creation process. Due to its length and importance, the full prompt is documented below.

3.2.2 Normalization and Multilingual Expansion

Following data generation, the unnormalized English corpus was transformed into standardized text through a normalization process that includes abbreviation expansion, numeral normalization, spelling correction, and punctuation standardization. Each noisy sentence was paired with its normalized counterpart to form a parallel dataset.

The normalized English data was subsequently translated into **11 additional Indian scheduled languages**, resulting in a total of **12 languages including English**. Translation was performed only after normalization to preserve semantic consistency across languages.

This pipeline results in a reliable multilingual dataset suitable for normalization and related NLP tasks.

System Prompt for Synthetic Generation for Normalization

You are tasked with generating synthetic data to train a normalization model in the Indian context. The data must be in English, reflecting Indian English conventions, and contain unnormalized elements such as abbreviations, numerals, dates, times, and category-specific terms. The output should be natural, coherent, and contextually relevant, incorporating Indian names, places, institutions, cultural references, and everyday scenarios. The generated text will be tailored to a specific category, subcategory, and document type, as specified below.

Input Parameters
-Category: {category}
(e.g., Medical, Legal, Numbers, Date & Time, Tech, Currency, Travel, Education, Finance, Identifiers, Units of Measurements, Titles, Addresses)
- Subcategory: {subcategory}
(e.g., for Medical: Diseases, Treatments, Hospitals; for Legal: Court Cases, Laws, Contracts)
- Document Type: {document_type}
(e.g., Twitter thread, Article, Comment, Quora post, Reddit post, Blog post, News snippet)
- Target Length: {target_length}
- Tone: {tone}
(e.g., "formal", "informal", "conversational", "professional")

Core Instructions
1. Objective:
Generate a {document_type} focused on the topic of {subcategory} within the {category} category. The text must serve as unnormalized training data for a normalization model, meaning all elements (e.g., abbreviations, numbers, dates) should remain in their raw, unstandardized forms.
2. Indian Context:
- Use Indian English (e.g., "prepone" instead of "bring forward", "lakh" instead of "100,000").
- Incorporate Indian proper nouns:
- Names (e.g., "Priya Sharma", "Rakesh Kumar", "Aisha Khan").
- Places (e.g., "Mumbai", "Delhi", "Kerala", "Guwahati").
- Institutions (e.g., "AIIMS", "IIT Bombay", "State Bank of India").
- Cultural references (e.g., "Diwali", "chai", "Bollywood", "saree").
- Reflect Indian societal norms, professions, and daily life (e.g., "auto-rickshaw driver", "government clerk", "monsoon season").

3. Unnormalized Data:
- Include multiple instances of unnormalized elements specific to the {category} and {subcategory}.
- Do not convert data into standardized forms (e.g., keep "Dr." instead of "Doctor", "15/08/2024" instead of "15th August 2024", "100" instead of "one hundred").
- Ensure variety in the types and formats of unnormalized data (e.g., mix abbreviations, numerals, and shorthand notations).
- Naturally integrate additional unnormalized elements from other categories where relevant (e.g., a Medical text might include dates or numbers alongside medical abbreviations).

4. Tone and Style:
Adapt the tone and style to the {document_type} and {tone} parameter:
- Article: Formal or semi-formal, detailed, 3-5 paragraphs, structured with an introduction, body, and conclusion.
- Comment: Brief, 1-2 sentences, conversational or opinionated, as if responding to a post or question.
- Quora post: Conversational, informative, 2-3 paragraphs, answering an implied question.
- Reddit post: Casual, engaging, 2-3 paragraphs, possibly with humor or personal anecdotes.
- Blog post: Personal or professional, 5-6 paragraphs, narrative-driven.
- News snippet: Objective, concise, 1-2 paragraphs, reporting a fact or event.
Prioritize {tone} while aligning with the document type.

Figure 2: System prompt used for generating curated unnormalized English data

Punctuation

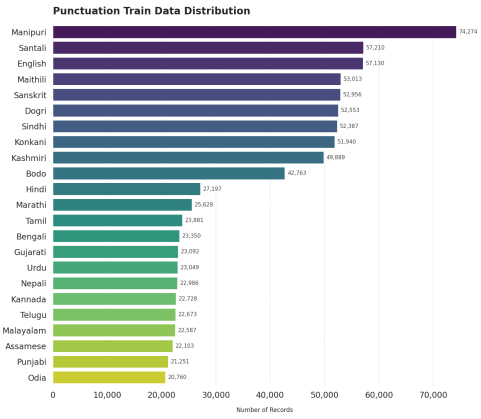


Figure 3: Punctuation data for each language

4 Methodology

This work develops a *unified* multilingual system that performs three complementary tasks at scale: **(i) transliteration** from native scripts to Latin; **(ii) text normalization** into human-readable form; and **(iii) punctuation restoration**. The system is built on two pillars: a *high-quality, carefully curated dataset* and a *rigorous multi-task training regimen*. Unless specified, the primary backbone is **Gemma-3-1B-PT** is used for efficiency studies.

Task Coverage. Transliteration covers **22** scheduled languages of India. Normalization is trained for **12** languages. Punctuation restoration is trained for **23** languages.

Data Preprocessing

Sources and splits. For transliteration, we ingest *existing* transliterated resources and additionally transliterate native-script corpora with *Gemini-2.0-Flash* to ensure full script coverage. For normalization, we construct (unnormalized, normalized) parallel pairs. For punctuation, we construct (unpunctuated, punctuated) pairs by removing punctuation from gold sentences. All datasets are split chronologically or by document (80/10/10) into train/dev/test.

We first normalize English, then translate normalized outputs into the target languages (12-language set) to create multilingual normalization supervision. We also build *merged pairs* that retain the original unnormalized text aligned to the normalized target.

Unified Problem Formulation

All three tasks are cast as conditional generation, *prompt-conditioned* rather than tag-conditioned. We prepend a short, task-specific natural-language prompt p to the input $\mathbf{x}$, and the model predicts the target sequence $y_{1:T}$:

$$P(y_{1:T} \mid \mathbf{x}, p) = \prod_{k=1}^{T} P(y_k \mid \mathbf{x}, p, y_{<k}) \, .$$

Here, p specifies the operation and (when needed) the target script:

- **Transliteration (to Latin):** e.g., ‘‘`Transliterate to Latin (ISO-compatible):`’’ $\|$ $\mathbf{x}$
- **Normalization (human-readable):** e.g., ‘‘`Normalize numbers, dates, addresses; produce human-readable text:`’’ $\|$ $\mathbf{x}$
- **Punctuation restoration:** e.g., ‘‘`Restore punctuation and casing faithfully:`’’ $\|$ $\mathbf{x}$

Targets are: Latin transliteration for transliteration prompts; normalized human-readable text for normalization prompts; and correctly punctuated text for punctuation prompts.

Model Architecture & Tokenization

We fine-tune **Gemma-3-1B-PT** (primary). Both are decoder-only Transformers with subword/byte-aware tokenization. We keep a shared vocabulary across tasks to maximize transfer. Lightweight output constraints are applied at decode time (below).

Training Objectives & Curriculum

Token-level objective. For a batch of N examples with targets of length T_i:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{T_i}\log P(y_{i,k} \mid \mathbf{x}_i, y_{i,<k})\,.$$

5 Evaluation Protocol

We evaluate the proposed unified multilingual text-processing system across three tasks: **transliteration (22 languages)**, **normalization (12 languages)**, and **punctuation restoration (23 languages)**. All experiments use consistent train–validation–test splits, and results are reported using both per-language and macro-averaged metrics.

5.1 Evaluation Metrics

Transliteration (22 Languages). We adopt four complementary metrics to comprehensively assess transliteration quality:

- **BLEU**: Measures n-gram overlap between predicted and reference transliterations.
- **ChrF++**: Character n-gram F-score capturing phonetic and spelling similarity.
- **Word Error Rate (WER)**: Measures word-level substitutions, insertions, and deletions.
- **Character Error Rate (CER)**: Strict character-level error measurement.

Together, these metrics capture sentence-level fidelity, phonetic accuracy, readability, and fine-grained character precision.

Normalization (12 Languages). Normalization performance is evaluated using:

- **CER** and **WER** against normalized references
- **Slot-level exact match accuracy** for structured entities such as numbers, dates, times, currencies, units, addresses, and abbreviations

Punctuation Restoration (23 Languages). We evaluate punctuation restoration using:

- Token-level **Precision**, **Recall**, and **F1-score**
- **BLEU** to measure fluency and sentence-level correctness

5.2 Baselines

For transliteration, we compare our unified model against the **Sarvam Transliteration Model**, a strong publicly available baseline for Indian language transliteration. Sarvam results are reported for eight overlapping languages using the same evaluation metrics (BLEU, ChrF++, WER, CER).

Normalization and punctuation restoration are evaluated against ground-truth references, as no comparable unified multilingual baselines are publicly available for these tasks.

6 Results

6.1 Transliteration Results

We evaluate transliteration performance using BLEU, ChrF++, WER, and CER. Results are reported separately for the Sarvam baseline model and our proposed unified multilingual model, followed by a direct comparison on overlapping languages.

6.1.1 Sarvam Transliteration Baseline

Table 1 presents the performance of the Sarvam transliteration model across **8 Indian languages**. While Sarvam serves as a strong baseline, its evaluation is limited to a smaller subset of languages.

Language	**BLEU**	**ChrF++**	**WER**	**CER**
Bengali	11.97	38.69	0.805	0.354
Gujarati	20.52	50.45	0.680	0.255
Hindi	37.55	65.19	0.473	0.324
Kannada	12.29	42.21	0.830	0.382
Malayalam	10.05	38.12	0.890	0.440
Marathi	15.93	49.36	0.699	0.247
Tamil	9.49	33.81	0.896	0.578
Telugu	11.24	40.01	0.880	0.426

Table 1: Sarvam transliteration model performance (baseline).

6.1.2 Unified Multilingual Model Results

Table 2 reports the transliteration performance of our **unified model across all 22 scheduled Indian languages**. The model demonstrates strong and consistent performance across diverse scripts and language families, including both high-resource and low-resource languages.

Language	**BLEU**	**ChrF++**	**WER**	**CER**
Assamese	14.71	42.84	0.714	0.293
Bengali	19.59	46.64	0.655	0.289
Bodo	13.38	50.71	0.690	0.213
Dogri	27.00	57.83	0.475	0.179
Gujarati	20.72	49.81	0.647	0.251
Hindi	49.47	73.25	0.337	0.168
Kannada	19.84	55.40	0.645	0.228
Kashmiri	11.92	42.36	0.696	0.276
Konkani	12.72	44.99	0.717	0.253
Maithili	25.82	60.55	0.539	0.163
Malayalam	13.79	44.56	0.771	0.321
Manipuri	16.81	47.12	0.697	0.285
Marathi	22.65	53.72	0.601	0.238
Nepali	17.71	54.18	0.620	0.196
Odia	15.21	51.34	0.670	0.215
Punjabi	21.48	48.98	0.595	0.258
Sanskrit	23.05	57.00	0.587	0.204
Santali	16.59	48.13	0.614	0.220
Sindhi	21.07	49.67	0.571	0.219
Tamil	14.47	45.37	0.743	0.317
Telugu	18.86	51.59	0.676	0.256
Urdu	30.13	57.95	0.486	0.210

Table 2: Unified model transliteration performance across 22 Indian languages.

6.1.3 Comparison with Sarvam Baseline

Table 3 provides a direct comparison between the Sarvam baseline and our unified model on overlapping languages. While Sarvam covers only 8 languages, our model achieves consistently superior performance while also scaling to a much larger language set.

Overall, the results clearly demonstrate that our unified model not only **outperforms the Sarvam baseline across all shared languages** but also **extends transliteration support from 8 to 22 Indian languages**. The consistent improvements in BLEU, along with reductions in WER and CER, confirm the effectiveness of unified multilingual and multi-task training for large-scale Indian language transliteration.

Language	Model	BLEU	WER	CER
Hindi	Sarvam	37.55	0.473	0.324
Hindi	Unified (Ours)	**49.47**	**0.337**	**0.168**
Bengali	Sarvam	11.97	0.805	0.354
Bengali	Unified (Ours)	**19.59**	**0.655**	**0.289**
Marathi	Sarvam	15.93	0.699	0.247
Marathi	Unified (Ours)	**22.65**	**0.601**	**0.238**
Tamil	Sarvam	9.49	0.896	0.578
Tamil	Unified (Ours)	**14.47**	**0.743**	**0.317**

Table 3: Direct comparison between Sarvam baseline and our unified model.

References

[1] Sumanth Doddapaneni, Rahul Aralikatte, Gowtham Ramesh, Shreya Goyal, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. Indiccorp v2: Monolingual corpora, benchmark and models for indic languages. `https://huggingface.co/datasets/ai4bharat/IndicCorpV2`, 2023. Accessed: 2025-11-12.

[2] Jay Gala, Pranjal A Chitale, A K Raghavan, Varun Gumma, Sumanth Doddapaneni, Aswanth M Kumar, Janki Atul Nawale, Anupama Sujatha, Ratish Puduppully, Vivek Raghavan, Pratyush Kumar, Mitesh M Khapra, Raj Dabre, and Anoop Kunchukuttan. Bpcc: Bharat parallel corpus collection — a comprehensive parallel corpus for all 22 scheduled indic languages. `https://huggingface.co/datasets/ai4bharat/BPCC`, 2023. Accessed: 2025-11-12.

[3] Mohammed Safi Ur Rahman Khan, Priyam Mehta, Ananth Sankar, Umashankar Kumaravelan, Sumanth Doddapaneni, Suriyaprasaad G, Varun Balan G, Sparsh Jain, Anoop Kunchukuttan, Pratyush Kumar, Raj Dabre, and Mitesh M. Khapra. Indicllmsuite: A blueprint for creating pre-training and fine-tuning datasets for indian languages. *arXiv preprint arXiv: 2403.06350*, 2024.

[4] Sidharth Pulipaka, Sparsh Jain, Ashwin Sankar, and Raj Dabre. Mark my words: A robust multilingual model for punctuation in text and speech transcripts, 2025.

[5] Microsoft Research. Updesh_beta: A multilingual question-answering and reasoning dataset. `https://huggingface.co/datasets/microsoft/Updesh_beta`, 2025. Accessed: 2025-11-12.